

ETS — A Readable Companion to the OTP Docs

Chapter 4 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why ETS exists

ETS is shared, in-memory storage owned by a process. It breaks the “share nothing” rule of chapter 2 in a controlled way: many processes can read one table without sending messages to anyone.

A `gen_server` that holds state in its loop is a bottleneck: every read waits in one mailbox. An ETS table lets readers go straight to the data, in constant time for a `set`, with no mailbox and no serialisation through a single process. That makes it the standard tool for caches, registries, counters and lookup tables.

What it does not remove is copying. Every insert copies the object into the table and every lookup copies it back out. ETS saves you the message round-trip, not the copy.

2. The building blocks

Table types

Type	Objects per key	Lookup cost	Notes
<code>set</code> (default)	1	Constant	Hash table
<code>ordered_set</code>	1	$O(\log N)$	Tree, keys in order; <code>1</code> and <code>1.0</code> are the same key
<code>bag</code>	Many, distinct	Grows with objects under that key	Avoid many objects per key
<code>duplicate_bag</code>	Many, duplicates allowed	Same as <code>bag</code>	Same warning

Access and ownership

Option	Who reads	Who writes
<code>protected</code> (default)	Any process	Owner only
<code>public</code>	Any process	Any process
<code>private</code>	Owner only	Owner only

Every table has one owner process. When the owner dies, the table is deleted, unless it has an `heir`, which then inherits it. `ets:give_away/3` hands a table to another living process. `named_table` lets you refer to it by atom instead of by table id.

Concurrency options

Option	Helps when	Cost
<code>read_concurrency</code>	Many parallel readers, bursts of reads	Switching between reads and writes gets more expensive
<code>write_concurrency</code> (true or auto)	Many processes writing different keys	More memory and per-operation overhead
<code>decentralized_counters</code>	Frequent inserts and deletes from many schedulers	<code>ets:info(T, size)</code> and memory become slower

These are tuning knobs, not correctness switches. Turn them on for a measured workload, not by default.

persistent_term: the read-only cousin

`persistent_term` is for data read constantly and changed almost never. Reads take no locks and do not copy the term. But every `put/2` or `erase/1` of a complex term starts a global garbage collection across all processes. The docs say plainly that it is “not a general replacement for ETS tables.”

3. How the runtime actually does it

What is atomic

Every update to a single object is atomic and isolated. A reader never sees half an update, and an update either happens completely or not at all. `update_counter` and `update_element` extend this to read-modify-write on fields of one object.

Nothing larger is atomic. There are no transactions across two keys, and a lookup followed by an insert is two separate operations that another process can interleave.

Traversal is not a snapshot

No traversal gives a consistent snapshot of a table that is being updated concurrently. Walking a `set` with `first/next` while others insert can skip keys, return a key twice, or raise `badarg`. Traversal is safe if:

- the table is an `ordered_set`, or
- the whole traversal is one ETS call (such as `ets:select/2`), or
- you wrap it in `ets:safe_fixtable/2`.

Keys decide the cost of a query

`lookup/2` with a known key is constant time on a `set` and $O(\log N)$ on an `ordered_set`. `match` and `select` whose pattern leaves the key unbound have to scan the entire table. An `ordered_set` can avoid the full scan when the pattern binds a prefix of the key, such as the first element of a tuple key.

4. Plausible but wrong

4.1 A table that dies with its creator

```
init([]) ->
  ets:new(sessions, [named_table, public, set]),
  {ok, #{} }.
%% handle_call/3 crashes on some bad input...
```

The `gen_server` owns the table. The first crash deletes every session, and processes reading `sessions` during the restart get `badarg`. The table's lifetime is tied to your least reliable code. Fix: give the table a dedicated owner that does nothing else, started earlier under the same supervisor, or set an `heir`. Decide deliberately whether the data should survive a crash; sometimes a fresh table is exactly what you want.

4.2 Read, then write

```
hit(Key) ->
  N = case ets:lookup(stats, Key) of
        [{_, C}] -> C;
        []       -> 0
      end,
  ets:insert(stats, {Key, N + 1}).
```

Two processes can read the same `N` and both write `N + 1`, losing one hit. Under load this undercounts silently. Fix: `ets:update_counter(stats, Key, 1, {Key, 0})`, which increments atomically and inserts the default if the key is missing.

4.3 Querying by a field that isn't the key

```
users_in(City) ->
  ets:match_object(users, {'_', '_', City, '_'}).
```

The key is unbound, so this scans every user on every call. It's fine in a test with 50 rows and becomes the slowest thing in production. `tab2list` plus a filter is the same scan with extra copying. Fix: keep a secondary index (bag of `{City, UserId}`), or choose a key that matches the common query. With an `ordered_set` keyed `{City, UserId}`, binding `City` avoids the full scan.

4.4 Walking a busy table by hand

```
expire(T) -> expire(T, ets:first(T)).
expire(_, '$end_of_table') -> ok;
expire(T, K) ->
  maybe_delete(T, K),
  expire(T, ets:next(T, K)).
```

On a `set` that other processes are writing, this can miss keys, visit some twice, or crash with `badarg` when `K` has been deleted. Fix: wrap the loop in `ets:safe_fixtable(T, true) / false`, or replace it with one `ets:select_delete/2` call using a match spec on the expiry field.

4.5 One giant object

```
ets:insert(config, {all, ConfigMap}),           %% 2 MB map
...
[{all, Cfg}] = ets:lookup(config, all),         %% on every request
Timeout = maps:get(timeout, Cfg).
```

Every request copies 2 MB to read one integer. Fix: store one object per setting (`{timeout, 5000}`), or, for configuration that changes almost never, `persistent_term:get({myapp, timeout})`, which does not copy.

4.6 persistent_term as a cache

```
remember(UserId, Profile) ->
    persistent_term:put({profile, UserId}, Profile).
```

Every `put` that replaces a complex term triggers a global GC that scans every process in the node. Used as a per-user cache, this stalls the whole system in proportion to how busy the cache is. Fix: ETS for anything that changes; `persistent_term` only for values set at startup or on rare configuration changes.

4.7 A bag with a hot key

```
ets:new(events, [bag, named_table, public]),
ets:insert(events, {Topic, Event}).           %% one popular Topic
```

Inserts and lookups on a `bag` get slower in proportion to the number of objects under the same key. The docs add that this linear search does not yield, so it also hurts the scheduling of other processes. Fix: add a unique part to the key (`{Topic, Seq}`) in an `ordered_set` and query by prefix.

5. Review checklist and sources

- ☐ Every table's owner is chosen deliberately (dedicated process or `heir`), and crash behaviour is intended
- ☐ Counters and other read-modify-write updates use `update_counter` / `update_element`
- ☐ Frequent queries use the key; other access paths have a secondary index
- ☐ No `tab2list` or unbound-key `match/select` on large tables in hot paths
- ☐ Traversals of concurrently updated tables use `ordered_set`, one ETS call, or `safe_fixtable`
- ☐ Objects are small enough to copy on every lookup
- ☐ `persistent_term` holds only rarely changed data
- ☐ Concurrency options are enabled for a measured reason
- ☐ `bag` tables have no hot keys

Sources

- ets module reference: <https://www.erlang.org/doc/apps/stdlib/ets.html>
- Tables and Databases, Efficiency Guide: <https://www.erlang.org/doc/system/tablesdatabases.html>
- persistent_term module reference: https://www.erlang.org/doc/apps/erts/persistent_term.html